Lab 7: Apache Kafka & Real-Time Messaging

Z5008 Big Data Lab

Stream Processing at Scale — East African Fintech Context

Dr. Innocent Nyalala

School of Engineering and Science
IIT Madras Zanzibar

Monday, 1 June 2026

Session Outline

Part I — Kafka Fundamentals

1. Kafka in 5 minutes (one end-to-end story)
2. Topics, partitions, offsets, keys
3. Consumer groups & rebalancing
4. Replication: leaders, followers, ISR
5. KRaft mode (control plane)

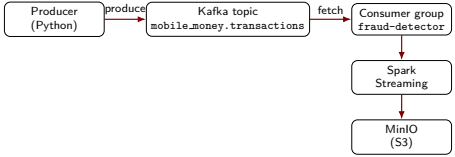
Part II — Lab Implementation

1. Docker services & Kafka UI (:8090)
2. Producer (JSON events) & Consumer (stats)
3. Topic management with AdminClient
4. Spark Structured Streaming from Kafka
5. Monitoring (lag) & key tuning knobs

Kafka One End-to-End Story

Scenario (mobile money)

- Producer emits events: `transaction_created`, `transaction_settled`, `fraud_flagged`
- Events go to one topic: `mobile_money.transactions`
- Key = `customer_id` (ordering per customer)
- Consumer group `fraud-detector` scales horizontally



What students should remember

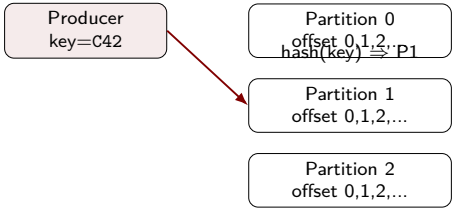
Producer → broker log → consumers. Offsets = "where I am" in the log.

Topic → Partitions: Where Events Actually Land

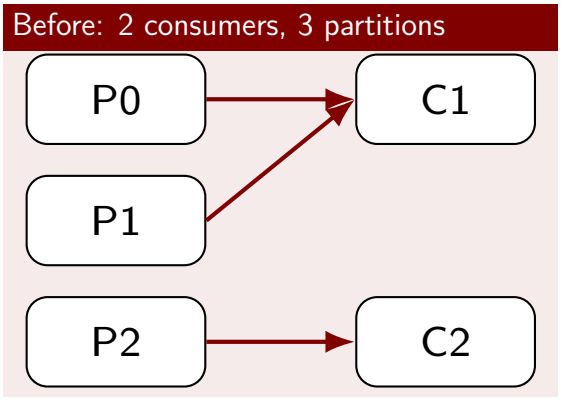
Key idea

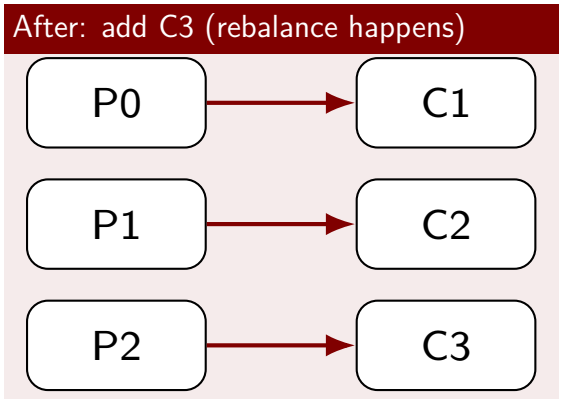
A topic is split into partitions (ordered logs). The **key** decides the partition.

- Same key ⇒ same partition ⇒ ordering for that key
- Different keys can be processed in parallel (different partitions)



Consumer Group Rebalancing (What Changes When You Scale)





What students should know

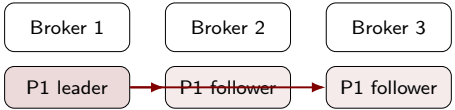
During rebalance, consumption pauses briefly; then partitions are reassigned.

Replication Failover: What Happens When a Broker Dies?

Story

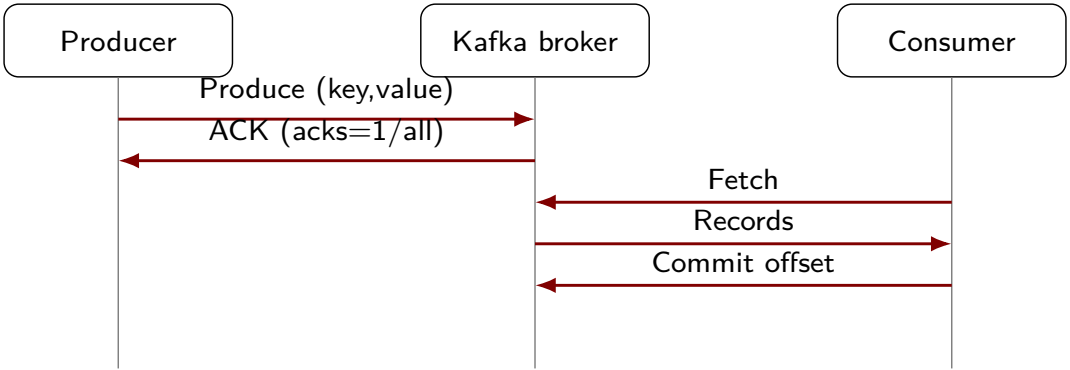
Partition leader handles reads/writes. Followers replicate. If leader fails, a follower becomes leader.

- producer acks depend on ISR (in-sync replicas)
- with `acks=all`, you trade latency for durability



if Broker 1 fails
⇒ elect new leader

Who Talks to Whom (Sequence View)



In KRaft mode

Controllers manage metadata and elections; producers/consumers still talk to brokers for data.

Why Event Streaming?

The East African Context

- M-Pesa, Tigo Pesa, Airtel Money: **billions** of mobile money events/day
- IoT sensors: weather stations, agricultural monitors, smart meters
- Fraud detection must act in **milliseconds**
- Traditional batch ETL $\Rightarrow$ 24-hour lag $\Rightarrow$ fraud goes undetected

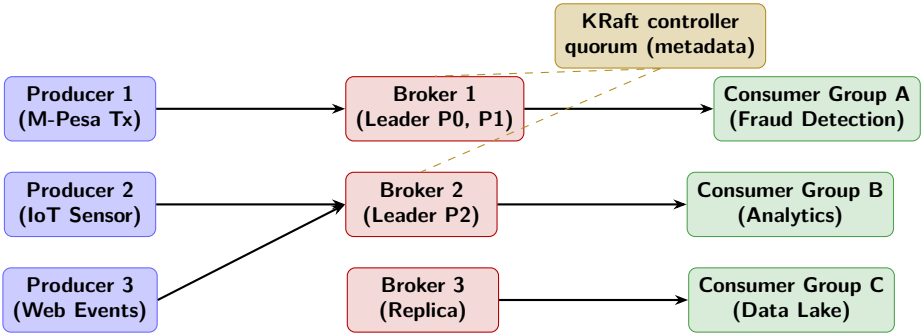
The Problem

How do you move **millions of events/second** reliably between dozens of microservices without tight coupling?

Apache Kafka Delivers

- Distributed **commit log**
- Persistent, replayable streams
- Horizontal scaling via partitions
- Decouples producers from consumers
- Throughput: 1M+ msgs/sec on modest hardware
- Battle-tested at LinkedIn, Netflix, Uber, Safaricom

Kafka Architecture — Core Components

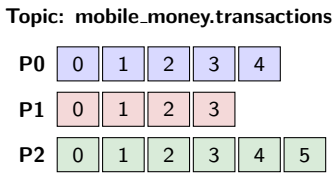


Key insight: Producers and consumers are completely decoupled — they never communicate directly.

Topics, Partitions, and Offsets

The Storage Model

- **Topic**: logical stream name (e.g., `mobile_money.transactions`)
- **Partition**: ordered, immutable sequence of records
- **Offset**: unique ID of a record within a partition (starts at 0)
- Records are **appended only** — never overwritten
- Consumers track their own offset — can replay at any time



Two consumer groups can read the same partition at different offsets.

Ordering Guarantee (and Why Keys Matter)

What Kafka Guarantees

- Ordering is guaranteed **within a partition**.
- Ordering is **not** guaranteed across partitions.

So What? Use a Key

- Set a **message key** (e.g., `customer_id`).
- Kafka hashes the key $\Rightarrow$ all events for that customer land in the same partition.
- You get per-customer ordering (critical for balances, fraud rules, etc.).

Example Keys

- `customer_id`
- `account_id`
- `merchant_id`

Rule of Thumb

If you need ordering, choose a key that defines the ordering unit.

Replication — Leader, Follower, ISR

Replication Concepts

- **Replication factor**: how many copies of each partition exist (typically 3)
- **Leader**: handles all reads & writes for a partition
- **Follower**: replicates from leader passively
- **ISR** (In-Sync Replicas): followers caught up within `replica.lag.time.max.ms`
- If leader fails, a new leader is elected from ISR
- `min.insync.replicas`: minimum ISR before a write is accepted

Example: RF=3, 3 Brokers

Partition	Leader	Followers
P0	Broker 1	Broker 2, 3
P1	Broker 2	Broker 1, 3
P2	Broker 3	Broker 1, 2

Why ISR Matters

With `acks=all` and `min.insync.replicas=2`, a write is only acknowledged after 2 replicas confirm receipt — **no data loss even on broker crash**.

KRaft Mode (Kafka Without ZooKeeper)

What changed?

Modern Kafka stores cluster metadata using an internal **Raft quorum** (KRaft), so ZooKeeper is not required.

What KRaft manages (control plane)

- broker membership and health
- topic + partition metadata
- leader election for partitions
- ACLs and cluster configuration

Important distinction

KRaft coordinates **metadata**. Your **messages** still live in broker partition logs.

Lab Setup (our Docker stack)

- Kafka broker: `kafka:9092`
- Kafka UI: `http://localhost:8090`
- No `zookeeper` service in this environment

Producer Internals

How a Producer Works

1. Serialize record (key + value) → bytes
2. Determine target partition (hash of key, or round-robin if no key)
3. Add to in-memory **RecordBatch** for that partition
4. Batch sent when `batch.size` reached OR `linger.ms` elapsed
5. Broker acknowledges based on `acks` setting
6. On error: retry up to `retries` times

Key Producer Configs

Config	Effect
`acks=0`	Fire & forget
`acks=1`	Leader ACK
`acks=all`	All ISR ACK
`batch.size`	16 KB default
`linger.ms`	0 ms default
`compression`	snappy/lz4
`retries`	2147483647

Tip

Set `linger.ms=5` + `batch.size=65536` for 3–5× throughput improvement with minor

Idempotent Producer & Exactly-Once Semantics

The Duplicate Problem

- Without idempotency: network timeout ⇒ producer retries ⇒ **duplicate records**
- Idempotent producer: each producer gets a **PID** (Producer ID) and sequence numbers
- Broker deduplicates retries automatically
- Enable: `enable.idempotence=True`
- Automatically sets `acks=all`, `retries=MAX_INT`, `max.in.flight=5`

Exactly-Once Semantics (EOS)

- Idempotent producer + **transactions**
- Atomic writes across topics/partitions
- Consumers read only **committed** records (`isolation.level=read_committed`)
- Cost: ∼10% throughput overhead

Delivery Guarantees (Producer + Consumer)

Summary

Guarantee	Producer config	Consumer config
At-most-once	`acks=0`	Auto-commit (risk of loss)
At-least-once	`acks=all` + retries	Manual commit *after* processing
Exactly-once	Idempotence + transactions	`isolation.level=read_committed`

Default recommendation for the lab

Use **at-least-once**: `acks=all` + manual commits.

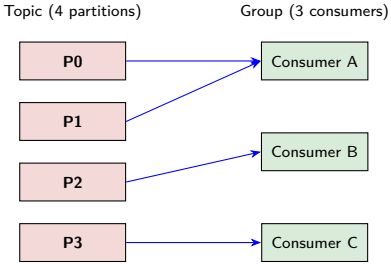
Python Producer — Mobile Money Events

```python
from kafka import KafkaProducer
import json, random, time, uuid

producer = KafkaProducer(
    bootstrap_servers=['kafka:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    key_serializer=lambda k: k.encode('utf-8'),
    acks='all',
    enable_idempotence=True,
    compression_type='snappy',
    batch_size=65536,
    linger_ms=5,
)

COUNTRIES = ['TZ', 'KE', 'UG', 'RW', 'ET']
TX_TYPES  = ['SEND', 'RECEIVE', 'WITHDRAW', 'DEPOSIT', 'PAY_BILL']

def generate_event():
    return {
        'transaction_id': str(uuid.uuid4()),
        'customer_id': f'CUST-{random.randint(1000, 9999)}',
        'country': random.choice(COUNTRIES),
        'amount': round(random.uniform(10, 500000), 2)
```

Consumer Groups & Partition Assignment

Consumer Group Model

- Multiple consumers sharing a `group.id` form a **consumer group**
- Each partition is assigned to **exactly one** consumer in the group
- Adding consumers scales throughput (up to partition count)
- Extra consumers beyond partition count sit **idle**
- Different groups each get **all** records (independent cursors)
- Offsets are committed to `__consumer_offsets` internal topic



Consumer A handles 2 partitions; B and C handle 1 each.

Consumer Configuration (Part 1)

Critical Consumer Configs

Config	Values / effect
`group.id`	Consumer group name
`auto.offset.reset`	`earliest` / `latest`
`enable.auto.commit`	`True` (risky) / `False`
`auto.commit.interval.ms`	5000 ms default
`max.poll.records`	500 default
`session.timeout.ms`	45000 ms
`heartbeat.interval.ms`	3000 ms

`auto.offset.reset`

- `earliest`: read from the start
- `latest`: read new events only
- Used only if the group has **no committed offset**

Consumer Configuration (Part 2)

Auto vs. Manual Commit

Auto-commit: commits every N ms even if processing hasn't finished. Risk: crash after poll, before processing ⇒ **data loss**.

Manual commit: commit *after* successful processing ⇒ **at-least-once delivery**.

Rebalancing

Triggered when: consumer joins/leaves group, or heartbeat timeout.
During rebalance, consumption is **paused**.

CooperativeSticky reduces disruption by only moving necessary partitions.

Python Consumer — Real-Time Statistics

```python
from kafka import KafkaConsumer
import json
from collections import defaultdict

consumer = KafkaConsumer(
    'mobile_money.transactions',
    bootstrap_servers=['kafka:9092'],
    group_id='fraud-analytics-group',
    auto_offset_reset='earliest',
    enable_auto_commit=False,
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),
)

count_by_country = defaultdict(int)
total_amount     = defaultdict(float)
total_count      = 0

try:
    for msg in consumer:
        event = msg.value
        country = event['country']
        count_by_country[country] += 1
        total_amount[country]     += event['amount']
```

Topic Management — Retention & Compaction

Retention Policies

- **Time-based** (`retention.ms`): delete records older than N ms. Default: 7 days (604 800 000 ms)
- **Size-based** (`retention.bytes`): delete oldest segments when total size exceeds limit. Per-partition.
- **Compaction** (`cleanup.policy=compact`): keep only the *latest* record per key. Ideal for **CDC**.
- Combined: `cleanup.policy=compact,delete`

Segment Files

AdminClient: Topic Parameters

Parameter	Recommendation
`num_partitions`	2× consumers
`replication_factor`	3 (prod) / 1 (lab)
`retention.ms`	86400000 (1 day)
`cleanup.policy`	delete or compact
`compression.type`	producer / snappy

Performance Tip

More partitions = more parallelism, but also more overhead. Rule of thumb: partitions = $\lceil$target throughput/single-partition throughput

Topic Management with AdminClient

```python
from kafka.admin import KafkaAdminClient, NewTopic
from kafka import KafkaConsumer

# Create AdminClient
admin = KafkaAdminClient(
    bootstrap_servers=['kafka:9092'],
    client_id='lab07-admin'
)

# Define new topic
topic = NewTopic(
    name='mobile_money.transactions',
    num_partitions=3,
    replication_factor=1,
    topic_configs={
        'retention.ms': '86400000',
        'cleanup.policy': 'delete',
        'compression.type': 'snappy',
    }
)

# Create (skip if already exists)
existing = admin.list_topics()
```

Kafka Connect — No-Code Integration

What is Kafka Connect?

- Framework for **scalable, reliable streaming** between Kafka and external systems
- No custom producer/consumer code needed
- **Source connectors**: pull data *into* Kafka (JDBC, FileStream, Debezium CDC)
- **Sink connectors**: push data *out* of Kafka (S3, MinIO, Elasticsearch, JDBC)
- Configuration via REST API or JSON files

MinIO Sink Connector Config

Key	Value
connector.class	S3SinkConnector
tasks.max	3
topics	mobile_money.transaction
s3.bucket.name	kafka-archive
s3.region	us-east-1
store.url	http://minio:9000
storage.class	S3Storage
format.class	JsonFormat
flush.size	1000

Deployment Modes

Connect vs. Custom Code

Use Connect for **standard integrations**

Schema Registry — Governance at Scale

The Schema Problem

- Without a schema: producer changes field name ⇒ consumer crashes
- **Schema Registry** (port 8081) is a central schema store
- Producers register an **Avro/JSON/Protobuf** schema before writing
- Consumers fetch schema by ID embedded in the message
- Enforces **compatibility rules** across versions

Key takeaway

A schema lets producers evolve safely without breaking consumers.

Schema Registry — Example Avro Schema

Avro Schema (v2 — backward compatible)

```
{
  "type": "record",
  "name": "MobileMoneyTx",
  "fields": [
    {"name":"tx_id","type":"string"},
    {"name":"customer_id","type":"string"},
    {"name":"country","type":"string"},
    {"name":"amount","type":"double"},
    {"name":"tx_type","type":"string"},
    {"name":"timestamp","type":"string"},
    {"name":"currency","type":"string","default":"TZS"}
  ]
}
```

The `currency` field (with default) was added in v2, so v1 consumers still work (BACKWARD compatible).

Schema Registry — Compatibility Modes

Compatibility Modes

BACKWARD	New consumers read old data (add optional fields)
FORWARD	Old consumers read new data (delete fields)
FULL	Both backward + forward
NONE	No checking (dev mode)

Lab default

Use BACKWARD unless you have a strong reason not to.

Kafka Streams — Core Concepts

KStream and KTable

- **KStream**: unbounded stream of records (events)
- **KTable**: changelog stream (latest value per key)
- **GlobalKTable**: replicated lookup table (for joins)

Common Operations

- `filter`, `map`, `flatMap`, `branch`
- Group/aggregate: `groupByKey().count()`
- Join/enrich: `join(ktable)`

Note for this lab

Kafka Streams runs in the JVM. For Python, we'll use **PySpark Structured Streaming**.

Kafka Streams — Windowing Types

Why Windows?

Streams never end, so we aggregate over time windows (per minute, per 5 minutes, etc.).

Window Types

Tumbling	Fixed, non-overlapping (e.g., count per minute)
Hopping	Fixed size, overlapping (e.g., 5-min window every 1 min)
Session	Gap-based, variable size (e.g., user sessions)
Sliding	Moves with event time (e.g., last 5 min)

In Spark

Use `window(...)` + watermarking to handle late events.

Spark Structured Streaming — Reading from Kafka

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import (
    StructType, StructField, StringType, DoubleType
)

spark = SparkSession.builder \
    .appName("KafkaMobileMoneyStream") \
    .master("spark://spark-master:7077") \
    .config("spark.jars.packages",
            "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0") \
    .getOrCreate()

schema = StructType([
    StructField("transaction_id", StringType()),
    StructField("customer_id",    StringType()),
    StructField("country",        StringType()),
    StructField("amount",         DoubleType()),
    StructField("tx_type",        StringType()),
    StructField("timestamp",      StringType()),
])

raw = spark.readStream \
```

Spark Structured Streaming — Windowed Aggregation

```python
from pyspark.sql.functions import (
    to_timestamp, window, sum as _sum, count, avg
)

events = parsed.withColumn(
    "event_time",
    to_timestamp(col("timestamp"), "yyyy-MM-dd'T'HH:mm:ss'Z'")
)

# 1-minute tumbling window aggregation by country
windowed = events \
    .withWatermark("event_time", "30 seconds") \
    .groupBy(
        window(col("event_time"), "1 minute"),
        col("country")
    ) \
    .agg(
        count("*").alias("tx_count"),
        _sum("amount").alias("total_amount"),
        avg("amount").alias("avg_amount")
    )

# Write to console (for lab demo)
```

Monitoring (Part 1) — Consumer Lag

What is Consumer Lag?

- Lag = latest offset − consumer committed offset
- Lag > 0: consumers are behind producers
- Growing lag $\Rightarrow$ scale consumers or optimise processing
- Offsets live in `__consumer_offsets`

CLI Tools

- `kafka-consumer-groups.sh --describe --group G`
- `kafka-topics.sh --list`
- `kafka-log-dirs.sh --describe`
- `kafka-dump-log.sh`

Rule of thumb

If lag grows steadily, you're under-provisioned.

Monitoring (Part 2) — Kafka UI

Kafka UI (Port 8090) shows

- Topic browser: messages + partitions
- Consumer group lag per partition
- Producer throughput (msgs/sec)
- Broker health (disk, network)
- Schema Registry (if enabled)

Key metric

Monitor **records-lag-max** per consumer group.

Alert idea

Alert if lag $> 10\times$ your target processing rate.

Performance Tuning (Part 1) — Producer + Topics

Producer Tuning

Goal	Setting
High throughput	`linger.ms=20`
	`batch.size=131072`
	`compression=lz4`
Low latency	`linger.ms=0`, `acks=1`
Durability	`acks=all`, `min.insync.replicas=2`

Topic Tuning

- Partitions $\approx 2\times$ peak consumer count
- **lz4**: speed **zstd**: compression
- Production: `replication.factor=3`

Performance Tuning (Part 2) — Consumers + Brokers

Consumer Tuning

Goal	Setting
Throughput	`fetch.min.bytes=50000`
	`max.poll.records=2000`
Latency	`fetch.min.bytes=1`
Memory	Limit `max.partition.fetch.byte`

Broker Tuning

- `num.io.threads`: match disk count
- `num.network.threads`: match CPU
- Use SSDs for `log.dirs`
- Leave RAM for page cache

Kafka Security (Part 1) — What to Enable

Security Layers

1. **Authentication**: SASL (SCRAM preferred)
2. **Encryption**: TLS on listeners
3. **Authorization**: ACLs

Typical choices

- SASL/SCRAM-SHA-256
- TLS everywhere
- Minimal ACLs

Lab vs. Production

Lab uses PLAINTEXT. Production fintech deployments should enable SASL + TLS.

Kafka Security (Part 2) — ACL Examples

Example ACL Commands

```
# Grant producer access
kafka-acls.sh --add \
  --allow-principal User:app1 \
  --operation Write \
  --topic mobile_money.transactions

# Grant consumer access
kafka-acls.sh --add \
  --allow-principal User:app2 \
  --operation Read \
  --topic mobile_money.transactions \
  --group fraud-group
```

Real-World Patterns (Part 1) — Event Sourcing & CQRS

Event Sourcing

- Store every state change as an immutable event in Kafka
- Rebuild state by replaying the log
- Log compaction can keep the latest value per key

CQRS

- Commands write events; queries read from materialised views
- Kafka becomes the single source of truth

Why it matters

Great fit for ledgers: auditability + replay.

Real-World Patterns (Part 2) — CDC & Fan-Out

CDC — Change Data Capture

- Debezium reads DB transaction logs (binlog/WAL)
- Each INSERT/UPDATE/DELETE becomes a Kafka event
- Downstream consumers see DB changes in real time

Fan-Out

One topic, many consumer groups:

- Fraud detection
- Analytics
- Data lake archival

Key idea

Producers and consumers stay decoupled.

Lab Exercise — Mobile Money Pipeline

Step-by-Step

1. **Verify environment:** `docker compose ps`, open Kafka UI at `http://localhost:8090`
2. **Create topic** `mobile_money.transactions` (3 partitions, RF=1) via AdminClient or Kafka UI
3. **Run producer:** send 1000 simulated mobile money events (JSON) with keys = `customer_id`
4. **Run consumer:** print running count per country, commit manually on each message

Services & Ports

Service	Address
JupyterLab	:8888
Kafka broker	kafka:9092
Kafka UI	:8090
Spark Master	`spark://spark-master:7077`
MinIO	http://minio:9000

Assessment

See **Week 7 Assignment**
Due: **Sunday 3 May 2026, 23:59 EAT**
100 marks across 5 tasks.

Session Summary (Part 1) — What We Covered

Topics

1. Kafka architecture: broker, topic, partition, offset, ISR
2. Producer: batching, compression, `acks`, idempotency
3. Consumer: groups, assignment, rebalancing, offsets
4. Topic management: retention, compaction, AdminClient
5. Connect + Schema Registry (overview)
6. Stream processing: Kafka Streams (concept), Spark Structured Streaming
7. Monitoring + security + real-world patterns

Session Summary (Part 2) — Remember These

Delivery Guarantees

At-most-once Fastest; loss possible (`acks=0`).

At-least-once No loss; duplicates possible (manual commit).

Exactly-once No loss + no duplicates (transactions).

Next Week — Lab 8

Advanced Spark Streaming: watermarking, stateful operations, checkpointing, Delta Lake / Apache Iceberg on MinIO.

Questions?

Office hours: Wednesday 14:00–16:00

References & Further Reading

Official Documentation

- Apache Kafka: `kafka.apache.org/documentation`
- confluent-kafka Python: `docs.confluent.io/kafka-clients/pyt`
- kafka-python: `kafka-python.readthedocs.io`
- PySpark Streaming + Kafka: `spark.apache.org/docs/latest/structured-streaming-kafka-integrat`
- Schema Registry: `docs.confluent.io/platform/current/schema-registry`

Books

- *Kafka: The Definitive Guide* — Shapira, Palino et al. (O'Reilly, 2nd ed.)
- *Designing Data-Intensive Applications* — Kleppmann (O'Reilly) — Ch. 11
- *Streaming Systems* — Akidau, Chernyak, Lax (O'Reilly)

Internal Resources

- Lab notebook: `notebooks/lab07_kafka.ipynb`
- Assignment: